# Introduction to SQL (Basic Concepts)

**Module Outcome:**

- Place a command in DDL / DML / DQL / DCL / TCL — not “SQL is SELECT”
- Create a small table and insert a row with MySQL quotes ( `'Ravi'` )
- Treat SQL as declaring *what* you want, not looping like Python

## Introduction to SQL [Basics]

- SQL stands for **Structured Query Language**.
- SQL is used to perform operations on Relational DBMS.
- SQL is declarative. Hence, easy to learn.
- SQL provides multiple clauses (commands) to perform various operations like create, retrieve, update and delete the data.

**Note:** This course uses **MySQL 8+** syntax. Use single quotes for string literals ( `'value'` ). See MySQL Setup and Connection.

## SQL Language Categories

SQL commands are grouped into five categories based on their purpose:

| Category | Full Form | Purpose | Commands |
|---|---|---|---|
| **DDL** | Data Definition Language | Define and modify database structure | `CREATE` , `ALTER` , `DROP` , `TRUNCATE` |
| **DML** | Data Manipulation Language | Insert, update, and delete data | `INSERT` , `UPDATE` , `DELETE` |
| **DQL** | Data Query Language | Retrieve data from tables | `SELECT` |
| **TCL** | Transaction Control Language | Manage transactions | `COMMIT` , `ROLLBACK` , `SAVEPOINT` |
| **DCL** | Data Control Language | Manage user access and permissions | `GRANT` , `REVOKE` |

```
DDL  →  Structure (tables, columns)
DML  →  Data changes (insert, update, delete)
DQL  →  Data retrieval (select)
TCL  →  Transaction safety (commit, rollback)
DCL  →  Security (grant, revoke)
```

**Note:** TCL and DCL are covered in Transactions and Indexes.

## DDL vs DML: Commit Behavior

In MySQL, **DDL** and **DML** commands behave differently when it comes to saving changes.

| | DDL | DML |
|---|---|---|
| **Commands** | `CREATE`, `ALTER`, `DROP`, `TRUNCATE` | `INSERT`, `UPDATE`, `DELETE` |
| **What it changes** | Database structure (tables, columns) | Data inside tables |
| **Auto-commit** | **Yes** — changes are saved immediately | **No** — changes are pending until you `COMMIT` |
| **ROLLBACK** | **Cannot** undo a completed DDL statement | **Can** undo uncommitted changes with `ROLLBACK` |

**Example — DDL is auto-committed:**

```
CREATE TABLE customers (
    name VARCHAR(60),
    mobile_number VARCHAR(15),
    account_id VARCHAR(20) PRIMARY KEY
);
-- Table is created immediately. There is nothing to ROLLBACK.
```

**Example — DML can be rolled back:**

```
START TRANSACTION;

UPDATE customers
SET mobile_number = '9998888567'
WHERE account_id = '123456';

-- Changes are NOT permanent yet
ROLLBACK;   -- undoes the UPDATE

-- Or use COMMIT; to save the changes permanently
```

> **Note:** `DROP TABLE` and `TRUNCATE TABLE` are DDL commands — once they run successfully, the structure or data they remove cannot be recovered with `ROLLBACK`. Use them carefully.

## Creating Tables

**Create Table** — Creates a new table in the database.

**Syntax:**

```
CREATE TABLE table_name (
    column1 type1,
    column2 type2,
    ....
);
```

Here, `type1` and `type2` in the syntax are the datatypes of `column1` and `column2` respectively.

**Example:** Create a player table to store the following details of players.

| column_name | data_type | Constraint |
|---|---|---|
| id | INT / INTEGER | PRIMARY KEY |
| name | VARCHAR(200) | |
| age | INT / INTEGER | |
| score | INT / INTEGER | |

```
CREATE TABLE player (
    id INT NOT NULL PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(200),
    age INT,
    score INT
);
```

A **primary key** uniquely identifies each row in a table. No two rows can have the same primary key value. `AUTO_INCREMENT` lets MySQL automatically assign the next `id` when we omit it in `INSERT`.

We can check the details of the created table using `DESCRIBE` or `SHOW COLUMNS`.

## MySQL Data Types

MySQL provides a rich set of data types. Choose the right type based on the data you store.

### Integer Types

| Type | Storage | Range | Use Case |
|---|---|---|---|
| `TINYINT` | 1 byte | -128 to 127 | Age, status flags |
| `SMALLINT` | 2 bytes | -32,768 to 32,767 | Small counts |
| `INT` / `INTEGER` | 4 bytes | ±2 billion | IDs, quantities |
| `BIGINT` | 8 bytes | ±9 quintillion | Large IDs, timestamps |

### Decimal and Floating-Point Types

| Type | Description | Use Case |
|---|---|---|
| `DECIMAL(p,s)` / `NUMERIC(p,s)` | Exact fixed-point (p=precision, s=scale) | **Money, prices** — no rounding errors |
| `FLOAT` | Approximate 4-byte floating point | Scientific calculations |
| `DOUBLE` | Approximate 8-byte floating point | Large approximate values |

**Example:**

```
price DECIMAL(10,2)   -- up to 99999999.99 (exact)
rating FLOAT          -- approximate (e.g., 4.567890)
```

```
salary DECIMAL(12,2)  -- money: 1234567890.12
```

**Note:** Use `DECIMAL` for money. `FLOAT` / `DOUBLE` are approximate and can cause rounding errors in financial calculations.

## String Types

| Type | Description | Use Case |
|---|---|---|
| `CHAR(n)` | Fixed-length string (padded with spaces) | Country codes (`'IN'`), fixed codes |
| `VARCHAR(n)` | Variable-length string (up to n chars) | Names, emails, titles |
| `TEXT` | Large variable-length text | Descriptions, comments |

### Example:

```
country_code CHAR(2)        -- always 2 chars: 'IN', 'US'
name VARCHAR(200)           -- variable: 'Virat', 'Rakesh Kumar'
description TEXT            -- long text
```

## Date and Time Types

| Type | Format | Description |
|---|---|---|
| `DATE` | `'YYYY-MM-DD'` | Date only |
| `TIME` | `'HH:MM:SS'` | Time only |
| `DATETIME` | `'YYYY-MM-DD HH:MM:SS'` | Date and time |
| `TIMESTAMP` | `'YYYY-MM-DD HH:MM:SS'` | Date/time; auto-updates on row change |

### Example:

```
birth_date DATE                         -- '1990-05-15'
login_time TIME                         -- '14:30:00'
created_at DATETIME DEFAULT CURRENT_TIMESTAMP
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
```

## Other Types

| Type | Description |
|---|---|
| `BOOLEAN` / `BOOL` | Stored as `TINYINT(1)` : 0 = FALSE, 1 = TRUE |
| `JSON` | Stores JSON documents (MySQL 8+) |

### JSON Example (MySQL 8+):

```
CREATE TABLE product_meta (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(200),
    attributes JSON
);

INSERT INTO product_meta (name, attributes)
VALUES ('Smart Watch', '{"brand": "Apple", "color": "black", "features": ["GPS", "HR"]}');

SELECT name, attributes->>'$.brand' AS brand FROM product_meta;
```

## Inspecting Table Structure

Use these MySQL commands to view table details:

```
DESCRIBE player;
-- or
SHOW COLUMNS FROM player;
```

**Example output:**

| Field | Type | Null | Key | Default | Extra |
|---|---|---|---|---|---|
| id | int | NO | PRI | NULL | auto_increment |
| name | varchar(200) | YES | | NULL | |
| age | int | YES | | NULL | |
| score | int | YES | | NULL | |

## Table Constraints

Constraints enforce rules on the data stored in a table. Common constraints:

| Constraint | Description |
|---|---|
| PRIMARY KEY | Uniquely identifies each row; cannot be NULL |
| NOT NULL | Column must always have a value |
| UNIQUE | All values in the column must be different |
| DEFAULT | Assigns a default value when none is provided |
| CHECK | Validates data against a condition |
| FOREIGN KEY | Links a column to a primary key in another table |

**Syntax:**

```
CREATE TABLE table_name (
    column1 type1 NOT NULL,
    column2 type2 UNIQUE,
    column3 type3 CHECK (column3 > 0),
```

```
    ...
);
```

**Example:** Create an `employee` table with constraints.

```
CREATE TABLE employee (
    id INT NOT NULL PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(200) NOT NULL,
    email VARCHAR(200) UNIQUE,
    salary DECIMAL(12,2) CHECK (salary > 0),
    status VARCHAR(20) DEFAULT 'active',
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

**Notes:**

- `NOT NULL` prevents inserting NULL into that column.
- `UNIQUE` ensures no two rows share the same value in that column.
- `DEFAULT` assigns a value automatically when INSERT omits that column.
- `CHECK` rejects rows that fail the condition (e.g., negative salary).
- Foreign keys are covered in Modeling Databases.

**Example — DEFAULT in action:**

```
INSERT INTO employee (name, email, salary)
VALUES ('Ravi', 'ravi@acme.com', 50000);
-- status defaults to 'active', created_at defaults to current timestamp
```

## Inserting Rows (DML)

`INSERT` clause is used to insert new rows in a table.

**Syntax:**

```
INSERT INTO
    table_name (column1, column2, ..., columnN)
VALUES
    (value1, value2, ..., valueN),
    (value1, value2, ..., valueN);
```

Any number of rows from 1 to n can be inserted into a specific table using the above syntax.

**Example:** Insert name, age and score of 2 players in the player table.

```
INSERT INTO
    player (name, age, score)
VALUES
    ('Rakesh', 39, 35),
    ('Sai', 47, 38);
```

Upon executing the above code, both the entries would be added to the player table.

```
SELECT *
```

```
FROM player;
```

**Output:**

| name | age | score |
|---|---|---|
| Rakesh | 39 | 35 |
| Sai | 47 | 38 |

## Possible Mistakes

**Mistake 1:** The number of values that we're inserting must match with the number of column names that are specified in the query.

```
INSERT INTO
    player (name, age, score)
VALUES
    ("Virat", 31);
```

**Output:** Error: 2 values for 3 columns

**Mistake 2:** We have to specify only the existing tables in the database.

```
INSERT INTO
    player_Information (name, age, score)
VALUES
    ("Virat", 31, 38);
```

**Output:** Error: no such table: player_Information

**Mistake 3:** Do not add additional parenthesis `()` post VALUES keyword in the code.

```
INSERT INTO
    player (name, age, score)
VALUES
    (("Rakesh", 39, 35), ("Sai", 39, 40));
```

**Output:** Error: 2 values for 3 columns

**Mistake 4:** While inserting data, be careful with the datatypes of the input values. Input value datatype should be same as the column datatype.

```
INSERT INTO
    player(name, age, score)
VALUES
    ('Virat', 30, 'Hundred');
```

**WARNING:** If the datatype of the input value doesn't match the column datatype, MySQL raises an error in strict mode (the default in MySQL 8+).

## Retrieving Data (DQL)

`SELECT` clause is used to retrieve rows from a table.

The database consists of a **player** table that stores the details of players who are a part of a tournament. The player table stores the name, age and score of players.

## Selecting Specific Columns

To retrieve the data of only specific columns from a table, add the respective column names in the SELECT clause.

**Syntax:**

```
SELECT
    Column1,
    Column2,
    ....
    columnN
FROM
    table_name;
```

**Example:** Fetch the name and age of the players from the player table.

```
SELECT
    name,
    age
FROM
    player;
```

**Output:**

| name | age |
|---|---|
| Virat | 32 |
| Rakesh | 39 |
| Sai | 47 |
| ... | ... |

## Selecting All Columns

Sometimes, we may want to select all the columns from a table. Typing out every column name, for every time we have to retrieve the data, would be a pain.

**Syntax:**

```
SELECT *
FROM table_name;
```

**Example:** Get all the data of players from the player table.

```
SELECT *
FROM player;
```

**Output:**

| name | age | score |
|---|---|---|
| Virat | 32 | 50 |
| Rakesh | 39 | 35 |
| Sai | 47 | 30 |
| ... | ... | ... |

## Selecting Specific Rows

We use `WHERE` clause to retrieve only specific rows.

**Syntax:**

```
SELECT *
FROM table_name
WHERE condition;
```

**Note:** WHERE clause specifies a condition that has to be satisfied for retrieving the data from a database.

**Example:** Get the details of the player whose name is 'Ram' from the player table.

```
SELECT *
FROM player
WHERE name = 'Ram';
```

**Output:**

| name | age | score |
|---|---|---|
| Ram | 47 | 30 |

## Update Rows (DML)

`UPDATE` clause is used to update the data of an existing table in database. We can update all the rows or only specific rows as per the requirement.

## Update All Rows

**Syntax:**

```
UPDATE
    table_name
SET
    column1 = value1;
```

**Example:**

```
UPDATE
    player
SET
    score = 100;
```

## Update Specific Rows

**Syntax:**

```
UPDATE
    table_name
SET
    column1 = value1
WHERE
    column2 = value2;
```

**Example:**

```
UPDATE
    player
SET
    score = 150
WHERE
    name = 'Ram';
```

## Delete Rows (DML)

`DELETE` clause is used to delete existing records from a table.

## Delete All Rows

**Syntax:**

```
DELETE FROM
    table_name;
```

**Example:** Delete all the rows from player table.

```
DELETE FROM
    player;
```

## Delete Specific Rows

**Syntax:**

```
DELETE FROM
    table_name
WHERE
    column1 = value1;
```

**Example:** Delete 'shyam' from the player table.

> **Note:** We can uniquely identify a player by name.

```
DELETE FROM
    player
WHERE
    name = 'shyam';
```

**Warning:** We cannot retrieve the data once we delete the data from the table.

## TRUNCATE (DDL)

`TRUNCATE` removes **all rows** from a table quickly. Unlike `DELETE`, it cannot use a `WHERE` clause and typically cannot be rolled back.

**Syntax:**

```
TRUNCATE TABLE table_name;
```

**Example:**

```
TRUNCATE TABLE player;
```

| DELETE | TRUNCATE |
|---|---|
| DML command | DDL command |
| Can use `WHERE` to delete specific rows | Removes all rows only |
| Slower for large tables (logs each row) | Faster (deallocates data pages) |
| Can be rolled back (in a transaction) | Cannot be rolled back in most cases |
| Triggers fire on DELETE | Triggers do not fire on TRUNCATE |

> **Note:** Use `DELETE` when you need conditional deletion or rollback. Use `TRUNCATE` when you need to empty an entire table quickly.

## DROP Table (DDL)

`DROP` clause is used to delete a table from the database.

**Syntax:**

```
DROP TABLE
```

```
    table_name;
```

**Example:**

```
DROP TABLE player;
```

## Alter Tables (DDL)

`ALTER` clause is used to add, delete, or modify columns in an existing table.

## Add Column

**Syntax:**

```
ALTER TABLE
    table_name
ADD
    column_name datatype;
```

**Example:** Add a new column `jersey_num` of type integer to the player table.

```
ALTER TABLE
    player
ADD
    jersey_num INT;
```

**Note:** Default values for newly added columns in the existing rows will be NULL.

## Rename Column

**Syntax:**

```
ALTER TABLE
    table_name RENAME COLUMN C1 TO C2;
```

**Example:** Rename the column `jersey_num` in the player table to `jersey_number`.

```
ALTER TABLE
    player RENAME COLUMN jersey_num TO jersey_number;
```

## Drop Column

**Syntax:**

```
ALTER TABLE
    table_name DROP COLUMN column_name;
```

**Example:**

```
ALTER TABLE
    player DROP COLUMN jersey_number;
```

**Note:** `ALTER TABLE ... DROP COLUMN` is supported in MySQL 8.0+.